

Comprendre votre configuration

À vérifier avant de commencer :

Langages acceptés

Agent Development Kit (ADK) est compatible avec plusieurs langages de programmation :

- **Python 3.11 ou version ultérieure** (abordé dans ce cours)
- **Java 17 ou version ultérieure** (consultez le [guide de démarrage rapide Java](#))

Python est utilisé pour les exemples de ce cours. Les concepts s'appliquent à tous les langages compatibles.

Logiciels requis (Python)

- **Python 3.11 ou version ultérieure** – ADK nécessite Python 3.11 ou version ultérieure.
 - Vérifiez votre version : `python --version` ou `python3 --version`.
 - Si nécessaire, téléchargez-le depuis python.org.
- **pip** : programme d'installation de packages Python (inclus dans Python).
 - Vérifiez la version de pip : `pip --version` ou `pip3 --version`.
- **Terminal/invite de commande** : accès à une interface de ligne de commande.
 - Mac : application Terminal
 - Windows : PowerShell ou invite de commande
 - Linux : votre terminal préféré

Compatibilité des systèmes

-  macOS
-  Windows 10/11
-  Linux (Ubuntu, Debian, Fedora, etc.)

Hypothèses

Dans ce guide, nous partons des principes suivants :

- Vous disposez d'un accès administrateur/sudo pour installer les packages.
- Vous connaissez les commandes de terminal de base.
- Vous pouvez créer et modifier des fichiers texte.
- Vous disposez d'une connexion Internet pour télécharger les packages.

Remarque : Toutes les commandes de ce guide présentent les variantes associées aux différents systèmes d'exploitation dès que nécessaire.

Configuration détaillée

Suivez ces étapes pour configurer votre environnement de développement ADK.

Étape 1 : Vérifier l'installation de Python

Vérifiez tout d'abord que vous avez installé Python 3.11 ou version ultérieure.

Vérifiez la version de Python :

```
python --version
```

ou

```
python3 --version
```

Résultat attendu : `Python 3.11.x` ou version ultérieure (par exemple, `Python 3.12.0`)

Si Python n'est pas installé ou si sa version est trop ancienne :

- Téléchargez-le depuis python.org.
- Installez Python 3.11 ou version ultérieure.
- Redémarrez votre terminal après l'installation.
- Relancez la vérification de la version.

Étape 2 : Créer un répertoire d'espace de travail

Créez un dossier dédié pour vos projets ADK :

```
mkdir adk-workspace  
cd adk-workspace
```

Fonction :

- Crée un répertoire nommé `adk-workspace`.
- Accède à ce répertoire.
- Regroupe tous vos projets d'agent au même endroit.

Utilisateurs Windows : ces commandes fonctionnent aussi bien dans PowerShell que dans l'invite de commande.

Étape 3 : Créer un environnement virtuel

Un environnement virtuel isole les dépendances de votre projet de l'installation Python de votre

système, ce qui évite les conflits entre différents projets.

Créez l'environnement virtuel :

```
python -m venv .venv
```

Si **python** ne fonctionne pas, essayez :

```
python3 -m venv .venv
```

Fonction :

- Crée un dossier nommé **.venv** qui contient un environnement Python isolé.
- Cet environnement possède son propre interpréteur Python et son propre espace de packages.

Activez l'environnement virtuel :

Sous Mac/Linux :

```
source .venv/bin/activate
```

Sous Windows (PowerShell) :

```
.venv\Scripts\Activate.ps1
```

Sous Windows (invite de commande) :

```
.venv\Scripts\activate.bat
```

Comment savoir s'il est activé :

- L'invite de votre terminal devrait maintenant afficher **(.venv)** au début.
- Exemple : **(.venv) user@machine:~/adk-workspace\$**

Important : Vous devez activer cet environnement virtuel chaque fois que vous ouvrez un nouveau terminal pour travailler sur vos projets d'agent.

Étape 4 : Installer ADK

Installez ADK (avec l'environnement virtuel activé) :

```
pip install google-adk
```

Éléments installés :

- Le framework ADK avec des abstractions d'agent ([LlmAgent](#), [Runner](#), etc.)
- Outils CLI (commande [adk](#))
- Dépendances pour utiliser les modèles Gemini de Google

Vérifiez l'installation :

```
adk --version
```

Résultat attendu : numéro de version, par exemple [1.0.0](#) ou ultérieur

Si [adk --version](#) échoue :

- Vérifiez que votre environnement virtuel est activé (vous devriez voir ([.venv](#)) dans l'invite du terminal).
 - Essayez de désactiver et de réactiver l'environnement virtuel : exécutez la commande [deactivate](#), puis réactivez-le.
 - Réinstallez l'environnement virtuel : [pip install --force-reinstall google-adk](#).
-

Étape 5 : Obtenir votre clé API

Votre agent doit avoir accès à un grand modèle de langage (LLM), qui correspond au composant "Modèle" du cours 1. Dans ce cours, nous utiliserons les modèles Gemini de Google.

Deux options s'offrent à vous :

Option A : API Gemini via Google AI Studio (recommandée pour l'apprentissage)

C'est l'option la plus simple pour vous lancer.

Étapes :

1. Accédez à [Google AI Studio](#).
2. Connectez-vous à l'aide de votre compte Google.
3. Cliquez sur **Créer une clé API**.
4. Copiez la clé API (elle ressemble à [AIzaSyC...](#)).

Avantages :

-  Configuration rapide (nécessite uniquement une clé API)
-  Version sans frais disponible

-  Convient parfaitement pour le développement et l'apprentissage

Option B : Gemini via Google Cloud Vertex AI (pour la production)

Choisissez cette option si vous avez besoin de fonctionnalités de niveau entreprise ou si vous effectuez un déploiement en production.

Prérequis :

- Compte Google Cloud
- Facturation activée (essai sans frais disponible)
- Un projet Google Cloud

Étapes :

1. Créez un [projet Google Cloud](#) si vous n'en avez pas encore.
2. Activez l'[API Vertex AI](#).
3. Installez la [gcloud CLI](#).
4. Authentifiez-vous.

```
gcloud auth application-default login
```

5. Notez votre **ID de projet** (visible dans la console Cloud).
6. Notez votre **emplacement** (par exemple, `us-central1`).

Avantages :

-  Fonctionnalités de niveau entreprise
-  Meilleur scaling et meilleurs quotas
-  Sécurité prête pour la production

Pour ce cours, nous vous recommandons l'option A (Google AI Studio) pour plus de simplicité.

Étape 6 : Créer votre premier projet d'agent

ADK fournit une commande permettant de créer un projet d'agent avec la structure appropriée.

Créez le projet :

```
adk create my_first_agent
```

Cela crée :

```
my_first_agent/  
├── agent.py      # Code de l'agent principal (vous le modifierez)  
├── __init__.py   # Initialisation du package Python  
└── .env          # Variables d'environnement (vous le modifierez par la suite)
```

Explication des fichiers :

- **agent.py** : c'est dans ce fichier que vous définirez votre agent à l'aide de code Python. Ce fichier rassemble le modèle, les outils et les composants d'orchestration que vous avez découverts dans le cours 1. Il contient la définition de votre **root_agent**.
- **__init__.py** : fichier d'initialisation du package Python qui importe votre module d'agent. Il est nécessaire pour qu'ADK puisse détecter votre agent.
- **.env** : fichier spécial pour stocker des informations sensibles telles que les clés API. ADK charge automatiquement ce fichier, ce qui permet de ne pas inclure les secrets dans votre code.

Accédez au projet :

```
cd my_first_agent
```

Vérifiez le code de l'agent par défaut :

Ouvrez **agent.py** dans votre éditeur de texte. L'écran qui s'affiche devrait ressembler à ce qui suit :

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='Un agent assistant serviable.',
    instruction='Tu es un assistant serviable.'
)
```

Ce que fait ce code :

- Il importe la classe **Agent** depuis ADK.
- Il crée un **root_agent** (l'agent principal qu'ADK recherche).
- Il le configure avec les paramètres suivants :
 - **model** : **gemini-2.5-flash** (le LLM qui alimente le raisonnement)
 - **name**: **root_agent** (identifiant obligatoire)
 - **description** : ce que fait l'agent (utilisée dans les systèmes multi-agents)
 - **instruction** : comportement attendu de l'agent

Il s'agit d'un agent fonctionnel minimal. Dans le module 2, vous apprendrez à personnaliser ces paramètres et à ajouter des outils.

Remarque : Dans ce cours, nous utilisons Python. Si vous utilisez Java, consultez le [guide de démarrage rapide Java](#) pour connaître la structure et la configuration spécifiques des projets Java.

Étape 7 : Configurer votre clé API

Configurez maintenant le fichier `.env` avec la clé API que vous avez obtenue à l'étape 5.

Ouvrez le fichier `.env` dans votre éditeur de texte.

Si vous utilisez Google AI Studio (option A) :

Remplacez le contenu par ce qui suit :

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=your-actual-api-key-here
```

Remplacez `your-actual-api-key-here` par votre clé API provenant de Google AI Studio.

Exemple :

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=AIzaSyC4lW...
```

Si vous utilisez Vertex AI (option B) :

Remplacez le contenu par ce qui suit :

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

Remplacez :

- `your-project-id` par l'ID de votre projet Google Cloud
- `us-central1` par l'emplacement de votre choix (si différent)

Exemple :

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=my-agent-project-123
GOOGLE_CLOUD_LOCATION=us-central1
```

Remarque importante concernant la sécurité :

-  N'intégrez jamais le fichier `.env` au contrôle des versions (Git).
- Le fichier `.env` doit figurer dans `.gitignore`
- Les clés API sont des identifiants sensibles. Traitez-les comme des mots de passe.

Étape 8 : Vérifier la configuration

Vérifions que tout fonctionne en exécutant l'interface Web ADK.

Démarrez l'interface Web ADK :

```
adk web
```

Résultat attendu :

```
INFO:      Started server process
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Fonction :

- Démarre un serveur Web local à l'adresse `http://localhost:8000`.
- Ouvre automatiquement un onglet de navigateur.
- Fournit une interface visuelle pour interagir avec votre agent.

Indicateurs de réussite : ☒ Aucun message d'erreur dans le terminal. ☒ Le navigateur ouvre l'interface ADK. ☒ L'UI Web s'affiche avec le nom de votre agent.

Si vous constatez des erreurs :

✗ "ModuleNotFoundError : aucun module nommé 'google.adk'"

- Votre environnement virtuel n'est pas activé.
- Correction : réactivez-le avec `source .venv/bin/activate` (Mac/Linux) ou `.venv\Scripts\Activate.ps1` (Windows).

✗ "Clé API non valide" ou erreurs d'authentification

- Vérifiez que votre fichier `.env` contient les bons noms de variables.
- Pour Google AI Studio : utilisez `GOOGLE_API_KEY` (et NON `GEMINI_API_KEY`).
- Vérifiez qu'il n'y a pas d'espaces superflus autour du signe `=`.
- Vérifiez que la clé API est correcte (copiez-la et collez-la à nouveau depuis Google AI Studio).

✗ "Commande 'adk' introuvable"

- L'environnement virtuel n'est pas activé ou ADK n'est pas installé.
- Correction : activez venv, puis exécutez `pip install google-adk`.

Arrêter le serveur Web : appuyez sur `Ctrl+C` dans votre terminal pour arrêter le serveur une fois les tests terminés.

Comprendre votre configuration

Faisons le lien entre ce que vous venez de créer et les concepts du cours 1.

Les trois composants de votre environnement

Rappel : **Agent = modèle + outils + orchestration**

1. Modèle (configuré à l'étape 7)

- Votre fichier `.env` configure l'accès à Gemini.
- Gemini est le LLM qui fournit le raisonnement et la prise de décision.
- C'est le "cerveau" qui comprend le langage et fait des choix.

2. Outils (à venir dans le module 3)

- Fonctions que votre agent peut appeler pour effectuer des actions.
- Exemples : effectuer des recherches sur le Web, lire des fichiers et envoyer des e-mails.
- Ils font le lien entre "savoir" et "faire".

3. Orchestration (fournie par ADK)

- Framework qui exécute la boucle de l'agent.
- Gère : percevoir → raisonner → agir → vérifier → répéter.
- Vous l'avez installée à l'étape 4 (`pip install google-adk`).

Interaction des fichiers

[agent.py](#) – Point de connexion de tous les éléments :

Vous avez vu le code de l'agent par défaut à l'étape 6. Faisons le lien avec les concepts du cours 1 :

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',          # Modèle : le moteur de raisonnement (cours 1 !)
    name='root_agent',                # Identité : identifiant requis
    description='Un agent serviable.', # Objectif : ce que fait cet agent
    instruction='Tu es serviable.'     # Comportement : comment il doit agir
    # Outils : vous les ajouterez dans le module 3
    # Orchestration : gérée automatiquement par la classe Agent
)
```

Reprenons :

- **Modèle** (`gemini-2.5-flash`) : LLM qui fournit le raisonnement et la prise de décision.
- **Outils** (non présentés pour l'instant) : fonctions que l'agent appelle pour effectuer des actions (module 3).
- **Orchestration** : la classe `Agent` exécute automatiquement la boucle Percevoir → Raisonner → Agir → Vérifier.

`.env` – Identifiants sécurisés :

- Conserve les clés API hors du code
- Chargé automatiquement par ADK
- Jamais intégré à Git

`__init__.py` – Initialisation du package :

- Transforme votre dossier en package Python
- Importe votre module d'agent
- Nécessaire pour qu'ADK détecte les agents

Remarque : ADK prend également en charge la configuration basée sur YAML (consultez [Configuration d'agent](#)) et Java (consultez le [guide de démarrage rapide Java](#)). Ce cours se concentre sur les agents basés sur du code Python.

Workflow de développement

1. **Modifiez `agent.py`** : définissez le comportement de votre agent.
2. **Exécutez `adk web`** : testez votre agent dans l'interface Web.
3. **Procédez par itérations** : apportez des modifications, actualisez l'agent et testez-le à nouveau.

Plus tard, vous découvrirez d'autres façons d'exécuter des agents :

- `adk run` : interaction basée sur le terminal
- `adk api_server` : déploiement en tant que service d'API